昇思 MindSpore

MindSpore Transformers

MindSpore Transformers 大模型系列课程

MindSpore Transformers LLM推理迁移与实践

主讲人：Ashley

昇思MindSpore研发工程师

MindSpore Transformers SIG Contributor

MindSpore Transformers 大模型系列课程

专题一：套件介绍

《MindSpore Transformers训推Mcore架构介绍》

《MindSpore Transformers环境安装与镜像制作》

专题二：LLM训练

《MindSpore Transformers LLM预训练与微调介绍与实践》

《MindSpore Transformers LLM数据预处理介绍与实践》

《MindSpore Transformers Safetensors权重详解》

《MindSpore Transformers断点续训介绍与实践》

《MindSpore Transformers训练在线监控介绍与实践》

《MindSpore Transformers训练高可用特性介绍》

专题三：LLM推理

前置课程 《 MindSpore Transformers推理介绍与实践》

《MindSpore Transformers & vLLM部署与评测》

专题四：高阶开发

《MindSpore Transformers & Megatron-LM训练精度比对》

《MindSpore Transformers & vLLM推理精度比对》

《MindSpore Transformers LLM训练迁移与实践》

《MindSpore Transformers LLM推理迁移与实践》

目录

1. MindSpore Transformers Mcore推理架构介绍

2. MindSpore Transformers Hugging Face模型迁移

3. MindSpore Transformers Hugging Face权重加载

4. MindSpore Transformers 模型迁移案例

1. MindSpore Transformers Mcore推理架构介绍

MindSpore Transformers大模型推理技术架构全新升级

离线推理 & 在线部署

- Hugging Face权重加载
- 配置化模型开发
- 注册机制一键部署

Models

Hugging Face模型配置

Hugging Face
配置原生加载：0 Day迁移

TransformerConfig
Layer API接口
GPTModel

Megaton-LM API对齐：Cell级精度比对

- DeepSeek-V3
- Qwen3/Qwen3-MoE
- Llama3.1/3/2
- GLM4/4.5
- ...

模块化模型接口（Module Spec模块化组装）

- GPTModel
- TransformerBlock
- TransformerLayer
- MultiTokenPrediction
- MoELayer
- MLP
- MultiLatentAttention
- FlashAttention
- ColumnParallelLinear
- RowParallelLinear
- VocabParallelEmbedding
- ...

多维混合并行

- 模型并行(TP)
- 数据并行(DP)
- 专家并行(EP)
- 流水线并行(PP)
- 序列并行(SP/CP)

服务化部署 | 榜单评测

vLLM-MindSpore Plugin/ SGLang-MindSpore Plugin

- 接口兼容
- 组件解耦
- 最小化侵入式修改

AISBench基准工具/ OpenCompass榜单评测

- 支持20+主流数据集
- 服务化数据集精度验证
- 延迟和吞吐性能压测

模型快速迁移 | 精度比对

- Hugging Face配置原生加载
- MCore接口精度对齐vLLM
- Hugging Face Tokenizer无缝对接
- 快速权重转换
- Hugging Face权重在线加载
- Cell级dump工具

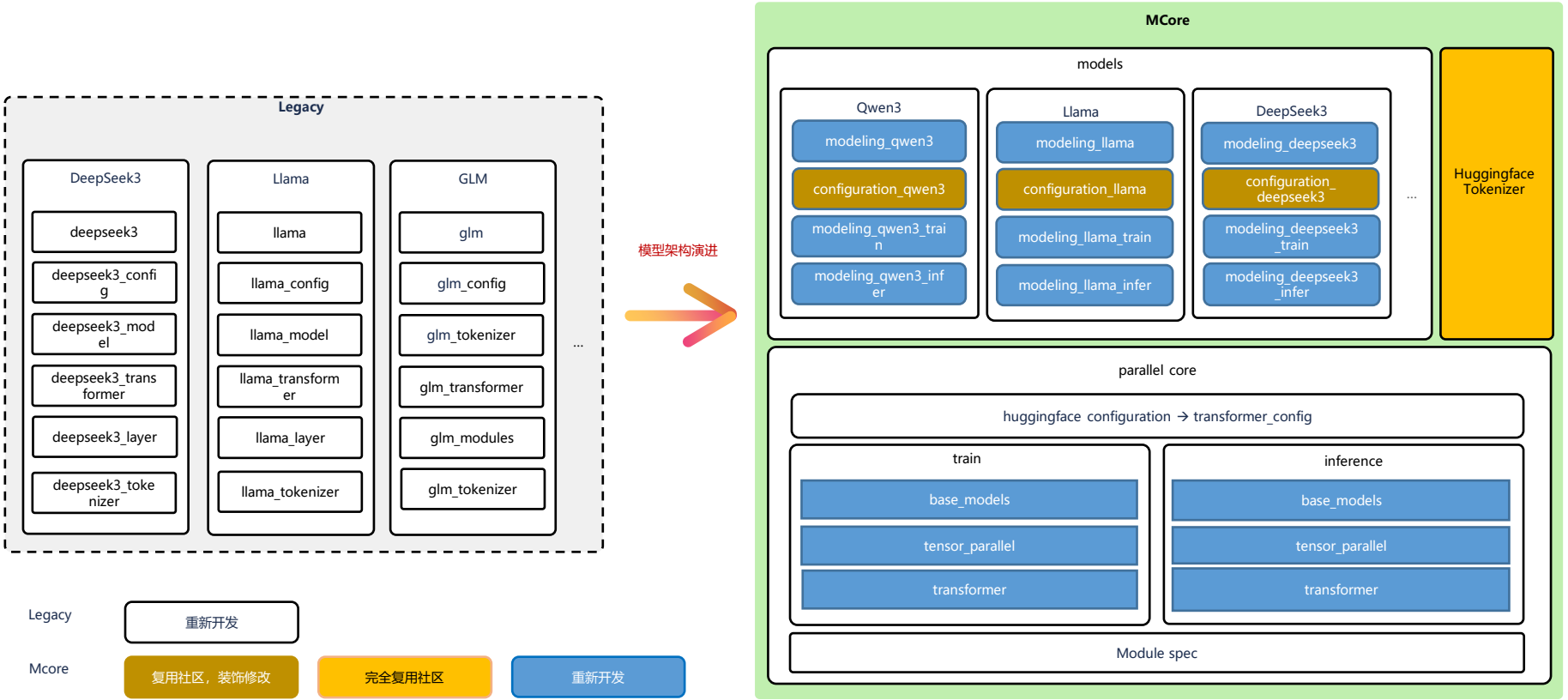
量化推理 | 高性能算子

- 支持金箍棒量化校准
- 支持HF/ModelSlim量化权重推理
- A16W8/A8W8/A8W4/C8...

- 对接昇腾ATB和ACLNN算子库
- 自研融合算子优化
- 自定义算子接入（能力构建中）

模型开箱即用 | **Hugging Face主流模型0day迁移** | **高精度/性能推理**

MindSpore Transformers Mcore模型库组件——整体架构



MindSpore Transformers Mcore推理模型库 ——整体架构

模型层 (parallel_core/inference/base_models)

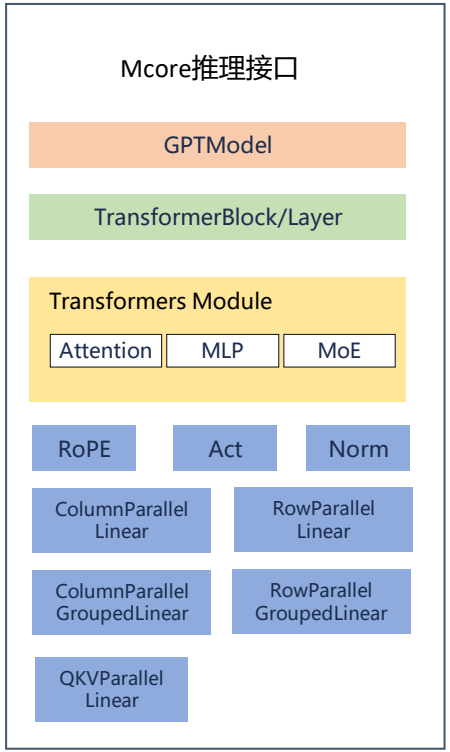
- 提供了预定义的主流 Transformer 模型架构。
- 主要子模块：
  - GPTModel: 用于实现类似 GPT 的自回归语言模型。
  - 模型层规范Spec构建: 用于实现大语言模型的泛化构建

Transformer 层 (parallel_core/inference /transformer)

- 这是整个库最核心的部分，它定义了 Transformer 架构的基本构建块，所有模型都是基于这个模块的组件构建的
- 提供模块化、可配置的 Transformer 组件。
- 主要子模块：
  - TransformerBlock/Layer： 代表一个完整的 Transformer 层
  - Attention/MLP/MOE： 实现多头注意力机制/前馈神经网络/专家模块，支持多种并行方式
  - LayerNorm/Activation/Rope：各种公共模块，归一化层、激活层、位置编码

分布式张量并行 (parallel_core/inference /tensor_parallel)

- 这是实现张量并行的核心，将模型层的权重在多张卡之间进行切分。
- 主要子模块：
  - ColumnParallelLinear/ColumnParallelGroupedLinear： 对线性层的权重矩阵按列切分。
  - RowParallelLinear/RowParallelGroupedLinear： 对线性层的权重矩阵按行切分。
  - QKVParallelLinear: 生成QKV矩阵，按照attention头进行切分



MindSpore Transformers Mcore推理模型库——Module架构

MLP

ColumnParallel Linear

RowParallel Linear

①定义数据类MLPSubmodules，表示MLP由哪些底层模块构成 （一个ColumnParallelLinear和RowParallelLinear）

②定义MLP类，实现__init__方法，传入子模块以及模块配置参数，定义模块结构

③实现MLP前向方法

```python
@dataclass
class MLPSubmodules:
    """
    The MLPSubmodules class defines two submodules for a Multi-Layer Perceptron (MLP):

    Args:
        linear_fc1 (Union[ModuleSpec, type], optional): The module definition for the first
            Defaults to None.
        linear_fc2 (Union[ModuleSpec, type], optional): The module definition for the second
            Defaults to None.
    """

    linear_fc1: Union[ModuleSpec, type] = None
    linear_fc2: Union[ModuleSpec, type] = None
```

```python
class MLP(nn.Cell):
    def __init__(
        self,
        config: TransformerConfig,
        submodules: MLPSubmodules,
        is_expert: bool = False,
        input_size: Optional[int] = None,
        delay_allreduce: bool = False,
        tp_group: Optional[ProcessGroup] = default_pgs,
        quant_config: Optional[QuantizationConfig] = None,
        prefix: str = "",
    ):
        super().__init__(config)

        self.config: TransformerConfig = config

        self.linear_fc1 = build_module(
            submodules.linear_fc1,
            self.input_size,
            ffn_hidden_size,
            config=self.config,
            bias=(self.config.add_bias_linear if self.config.add_mlp_fc1_bias_linear
                  is None else self.config.add_mlp_fc1_bias_linear),
            is_expert=is_expert,
            transpose_b=True,
            compute_dtype=self.config.compute_dtype,
            quant_config=quant_config,
            prefix=f"{prefix}.linear_fc1",
            **fc1_parallel_kwargs,
        )

        self.linear_fc2 = build_module(
            submodules.linear_fc2,
            self.config.ffn_hidden_size,
            self.config.hidden_size,
            config=self.config,
            bias=(self.config.add_bias_linear if self.config.add_mlp_fc2_bias_linear
                  is None else self.config.add_mlp_fc2_bias_linear),
            is_expert=is_expert,
            transpose_b=True,
            compute_dtype=self.config.compute_dtype,
            quant_config=quant_config,
            prefix=f"{prefix}.linear_fc2",
            **fc2_parallel_kwargs,
        )
```

MindSpore Transformers Mcore推理模型库——Module架构

SelfAttention

①定义数据类SelfAttentionSubmodules,表示SelfAttention由哪些底层模块构成

②定义SelfAttention类，实现__init__方法，传入子模块以及模块配置参数，定义模块结构

③实现前向方法

```python
@dataclass
class SelfAttentionSubmodules:
    """Configuration class for specifying the submodules of a self-attention."""

    linear_qkv: Union[ModuleSpec, type] = None
    core_attention: Union[ModuleSpec, type] = None
    linear_proj: Union[ModuleSpec, type] = None
    q_layernorm: Union[ModuleSpec, type] = None
    k_layernorm: Union[ModuleSpec, type] = None
```

```python
if self.use_flash_attention:
    self.core_attention = build_module(
        submodules.core_attention,
        head_num=self.num_attention_heads_per_partition,
        head_dim=self.hidden_size_per_attention_head,
        kv_head_num=self.num_query_groups_per_partition,
        scale_value=1.0 / self.norm_factor,
        next_tokens=0,
        pa_kv_head_num=self.num_query_groups_per_partition,
    )
```

```python
class SelfAttention(Attention):
    def __init__(self,
                 config: TransformerConfig,
                 submodules: SelfAttentionSubmodules,
                 layer_number: int,
                 attn_mask_type: AttnMaskType = None,
                 cp_comm_type: str = None,
                 delay_allreduce: bool = False,
                 model_comm_pgs: Optional[ModelCommProcessGroups] = default_model_comm_pgs,
                 quant_config: Optional[QuantizationConfig] = None,
                 prefix: str = ""):
        super().__init__(
            config=config,
            submodules=submodules,
            layer_number=layer_number,
            attn_mask_type=attn_mask_type,
            attention_type=None,
            cp_comm_type=cp_comm_type,
            delay_allreduce=delay_allreduce,
            model_comm_pgs=model_comm_pgs,
            quant_config=quant_config,
            prefix=prefix
        )

        self.linear_qkv = build_module(
            self.submodules.linear_qkv,
            self.config.hidden_size,
            self.hidden_size_per_attention_head,
            self.num_heads,
            self.config.num_query_groups,
            config=self.config,
            bias=self.config.add_qkv_bias,
            gather_output=False,
            transpose_b=True,
            compute_dtype=self.compute_dtype,
            tp_group=self.tp,
            quant_config=quant_config,
            prefix=f"{prefix}.linear_qkv",
        )
```

2. MindSpore Transformers Hugging Face模型迁移

MindSpore Transformers Hugging Face模型迁移——文件结构

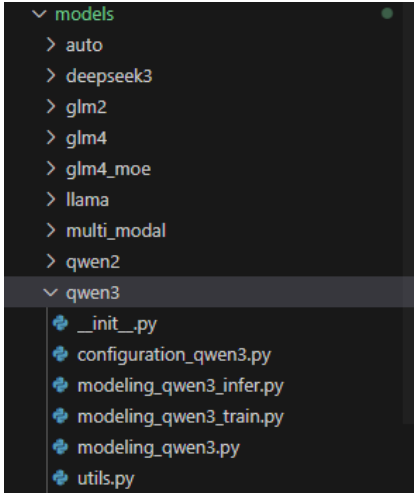
迁移总体流程

文件结构

模型代码放置于mindformers/models下，每个模型有独立文件夹

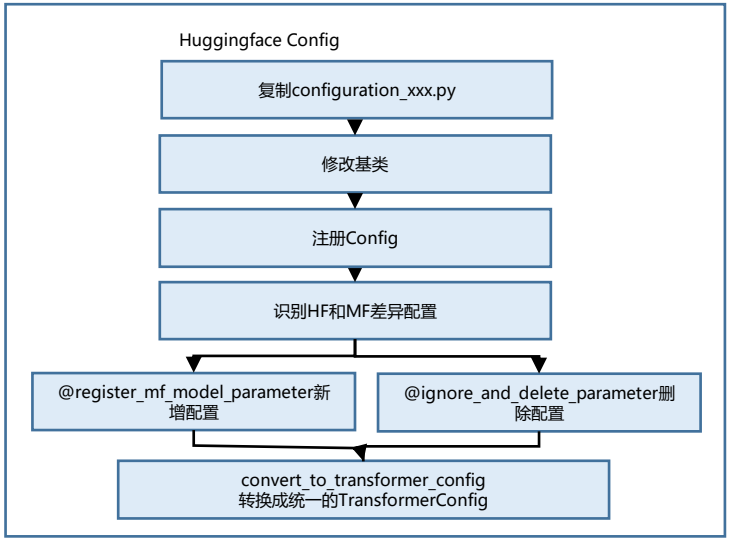
迁移HF模型时，主要需要创建以下文件：

- __init__.py
- configuration_xxx.py ---模型配置类
- modeling_xxx.py ---模型类
- modeling_xxx_infer.py ---推理模型类
- modeling_xxx_train.py ---训练模型类 （可选）
- utils.py



MindSpore Transformers Hugging Face模型迁移——HFConfig适配

1. 在configuration_xxx.py文件中，实现ModelConfig类。




```python
from mindformers.models.configuration_utils import PretrainedConfig
from mindformers.models.model_config_utils import (
    register_mf_model_parameter,
    ignore_and_delete_parameter,
    NotSupportedInfo
)
from mindformers.parallel_core.mf_model_config import MFModelConfig
from mindformers.tools.register import MindFormerRegister, MindFormerModuleType


zyw_hw, 3 months ago | 5 authors (sunyu-xuan and others)
@MindFormerRegister.register(MindFormerModuleType.CONFIG, legacy=False, search_names='qwen3')
class Qwen3Config(PretrainedConfig):
    """
    This is the configuration class to store the configuration of a [`Qwen3Model`]. It is used to instantiate a
    Qwen3 model according to the specified arguments, defining the model architecture. Instantiating a configuration
```

注册

修改基类

```python
@register_mf_model_parameter(
    mf_model_kwargs=MFModelConfig(
        pad_token_id=151643,
        block_size=32,
        num_blocks=1024,
        normalization='RMSNorm',
        add_bias_linear=False,
        gated_linear_unit=True,
        use_contiguous_weight_layout_attention=False
    ))
@ignore_and_delete_parameter(extra_ignore_param=[
    ('max_window_layers', NotSupportedInfo.useless),
    ('sliding_window', NotSupportedInfo.useless),
    ('use_sliding_window', NotSupportedInfo.useless),
    ('layer_types', NotSupportedInfo.useless),
])
def __init__(self,
             vocab_size=151936,
             hidden_size=4096,
             intermediate_size=22016,
             num_hidden_layers=32,
             num_attention_heads=32,
```

新增MF配置

删除配置

HuggingFace-Config	类型和默认值	MindFormers-Config（映射到TransformerConfig参数名称）
mlp_bias	bool: false	暂不支持
mlp_only_layers	list: []	暂不支持
sliding_window	null	暂不支持
output_router_logits	bool: false	暂不支持
router_aux_loss_coef	bool: true	暂不支持
decoder_sparse_step	int: 1	暂不支持
aux_loss_alpha	float: 0.001	暂不支持
seq_aux	bool: true	暂不支持
rope_scaling	"low_freq_factor": float,	不支持
	"high_freq_factor": float	不支持

HuggingFace-Config	类型和默认值	MindFormers-Config（映射到TransformerConfig参数名称）
use_cache	bool: true	use_past
torch_dtype	str: "bfloat16"	compute_dtype
max_window_layers	int	暂不支持
use_sliding_window	bool: false	暂不支持
quantization_config	dict:	暂不支持
	"activation_scheme": "dynamic"	暂不支持
	"fmt": "e4m3"	暂不支持
	"quant_method": "fp8"	暂不支持
	"weight_block_size"	暂不支持

MindSpore Transformers Hugging Face模型迁移——模型适配

2. 在utils.py文件中，实现模型基类。

①实现模型基类xxxPretrainedModel：继承PretrainedModel和ModelMixin类

② 指定config_class属性为模型对应Config类

```python
"""Qwen3 models' utils."""
from mindformers.models.qwen3.configuration_qwen3 import Qwen3Config
from mindformers.models.modeling_utils import PreTrainedModel
from mindformers.parallel_core.utils.model_mixin import ModelMixin
      zxq, 5 months ago • 【dev】【inference】qwen3_mcore模型适配

zxq, 3 months ago | 1 author (zxq)
class Qwen3PreTrainedModel(PreTrainedModel, ModelMixin):
    """
    An abstract class to handle weights initialization and a simple interface for downloading and loading pretrained
    models.
    """

    config_class = Qwen3Config
    base_model_prefix = "Qwen3"
```

继承基类

指定config类

MindSpore Transformers Hugging Face模型迁移——模型适配

2. 在modeling_xxx.py文件中，实现模型类。

① 实现最外层模型类：继承xxxPretrainedModel

② 注册模型

③ 实现__new__方法，根据不同的RUN_MODE，返回不同的模型

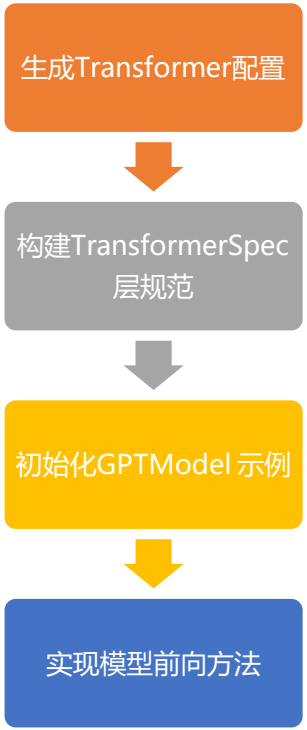
```
JavaZero, 4 months ago | 3 authors (zxq and others)
@MindFormerRegister.register(MindFormerModuleType.MODELS, legacy=False)
class Qwen3ForCausalLM(Qwen3PreTrainedModel):
    r"""
    Provide Qwen3 Model for training and inference.
    Args:
        config (Qwen3Config): The config of Qwen3 model.

    Returns:
        Tensor, the loss or logits of the network.
    """

    def __new__(cls, config):
        r"""
        get run mode to init different model.

        Args:
            config (Qwen3Config): The config of qwen3 model.

        Raises:
            NotImplementedError: Train mode is not supported for Qwen3 model.

        Returns:
            Tensor, the loss or logits of the network

        """
        if os.environ.get("RUN_MODE") == "predict":
            return InferenceQwen3ForCausalLM(config=config)
        return TrainingQwen3ForCausalLM(config=config)
```

注册模型

实现__new__方法

MindSpore Transformers Hugging Face模型迁移——模型适配

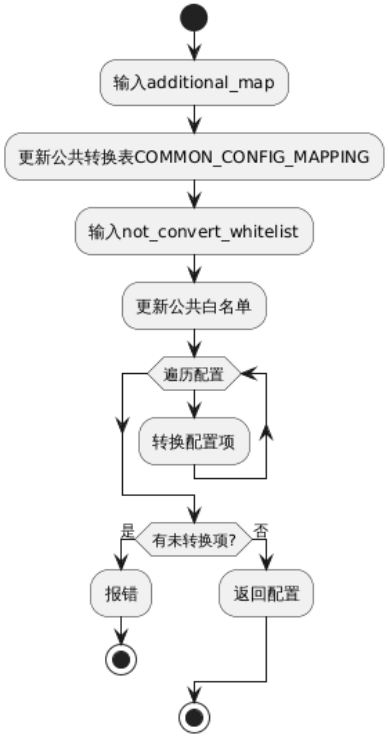
3、在modeling_xxx_infer.py文件中，实现推理模型类




```python
class InferenceQwen3ForCausalLM(Qwen3PreTrainedModel, InferModelMixin):

    def __init__(self, config: Qwen3Config):
        super().__init__(config, auto_prefix=False)
        self.config = config
        # step1: 由模型config转换生成transformer config
        config: TransformerConfig = self.convert_to_transformer_config(self.config)
        # 可选 量化推理配置
        self.quant_config = get_quant_config(self.config)
        # step2: 构建TransformerSpec层规范
        transformer_layer_spec=get_gpt_layer_local_spec(
                                normalization=config.normalization,
                                use_flash_attention=self.config.use_flash_attention,
                                qk_layernorm=True,
                            )
        # step3: 初始化GPTModel实例
        self.model = GPTModel(
            config=config,
            transformer_layer_spec=transformer_layer_spec,
            vocab_size=self.vocab_size,
            max_sequence_length=self.max_position_embeddings,
            position_embedding_type=config.position_embedding_type,
            rotary_base=self.config.rope_theta,
            share_embeddings_and_output_weights=self.config.tie_word_embeddings,
            pre_process=config.pre_process,
            post_process=config.post_process,
            quant_config=self.quant_config
        )
    # step4: 实现模型前向方法
    def construct(self, input_ids, hidden_states=None):
        logits = self.model(input_ids=input_ids,hidden_states=hidden_states)
        return logits
```

MindSpore Transformers Hugging Face模型迁移——模型适配

convert_to_transformer_config()的工作流



公共转换表 COMMON_CONFIG_MAPPING

位置： mindformers\parallel_core\transformer_config_utils.py

```
# Model Architecture
("mtp_depth", "num_nextn_predict_layers", "mtp_num_layers"): "mtp_num_layers",
("mtp_loss_factor", "mtp_loss_scaling_factor"): "mtp_loss_scaling_factor",
("num_heads", "num_attention_heads", "n_head"): "num_attention_heads",
("n_kv_heads", "num_key_value_heads", "num_query_groups"): "num_query_groups",
("intermediate_size", "ffn_hidden_size"): "ffn_hidden_size",
("head_dim", "kv_channels"): "kv_channels",
```

使用示例

```
"""
假设XXXConfig存在特殊配置。

覆盖公共转换：mtp_layers, compute_in_fp32

无用配置：kv_cache, top_k
"""

def is_fp32(compute_in_fp32):
  if compute_in_fp32:
    return float32
  return bfloat16

class TrainingXXXForCausalLM(TrainModelMixin, XXXPreTrainedModel):
    def __init__(self, config: XXXConfig):
        super().__init__(config, auto_prefix=False)

        transformer_config = self.convert_to_transformer_config(config,
                                                                 is_mla_model=True,
                                                                 additional_map={"mtp_layers": "mtp_num_layers",
                                                                                 "compute_in_fp32": ("compute_dtype", is_fp32)
                                                                                 },
                                                                 not_convert_whitelist={"kv_cache", "top_k"}
                                                                )
```

MindSpore Transformers Hugging Face模型迁移——模型适配

- Transformer大模型在模型结构上具有相似性，MCore通过以下两个核心函数实现大语言模型的泛化构建：

✓ get_gpt_layer_local_spec: 构建单个 Transformer 层的模块规范，适用于每层结构相同的模型搭建（如Qwen3）

✓ get_gpt_decoder_block_spec: 构建整个解码器块的模块规范，即多层Transformer模块，支持MoE、Dense混合层构建，适用于存在不同Transformer层结构的模型搭建（如DeepSeekV3）

- ModuleSpec是模型组装的实现类，可以通过指定module和submodules参数来动态创建我们想要的层

get_gpt_layer_local_spec

该函数用于定义单个Transformer层的模块规范，支持多种结构特性：

- MLP（Multilayer Perceptron）
- MoE（Mixture of Experts）
- GQA/MHA
- Multi Latent Attention
- Q/K LayerNorm
- Sandwich Norm

```
self_attn = ModuleSpec(
    module=SelfAttention,
    submodules=SelfAttentionSubmodules(
        core_attention=FlashAttention if use_flash_attention else DotProductAttention,
        linear_proj=RowParallelLinear,
        linear_qkv=QKVParallelLinear,
        q_layernorm=get_norm_cls(normalization) if qk_layernorm else IdentityOp,
        k_layernorm=get_norm_cls(normalization) if qk_layernorm else IdentityOp,
    )
)

mlp = ModuleSpec(
    module=MLP,
    submodules=MLPSubmodules(
        linear_fc1=MergedColumnParallelLinear if gated_linear_unit else ColumnParallelLinear,
        linear_fc2=RowParallelLinear,
    ),
)

transformer_layer_spec = ModuleSpec(
    module=TransformerLayer,
    submodules=TransformerLayerSubmodules(
        input_layernorm=get_norm_cls(normalization),
        self_attention=self_attention,
        post_self_attn_layernorm=get_norm_cls(normalization) if sandwich_norm else IdentityOp,
        pre_mlp_layernorm=get_norm_cls(normalization),
        mlp=mlp,
        post_mlp_layernorm=get_norm_cls(normalization) if sandwich_norm else IdentityOp,
    )
)
```

MindSpore Transformers Hugging Face模型迁移——模型适配

- Transformer大模型在模型结构上具有相似性，MCore通过以下两个核心函数实现大语言模型的泛化构建：

✓ get_gpt_layer_local_spec: 构建单个 Transformer 层的模块规范，适用于每层结构相同的模型搭建（如Qwen3）

✓ get_gpt_decoder_block_spec: 构建整个解码器块的模块规范，即多层Transformer模块，支持MoE、Dense混合层构建，适用于存在不同Transformer层结构的模型搭建（如DeepSeekV3）

get_gpt_decoder_block_spec

该函数用于构建整个 Transformer解码器块结构，支持：

- MoE和Dense层混合搭建
- 读取配置信息定义层间排布

```python
def get_gpt_decoder_block_spec(
        config: TransformerConfig,
        normalization: Optional[str] = None,
        qk_l2_norm: Optional[bool] = False,
) -> TransformerLayerSubmodules:
    """GPT block spec."""
    # layer specs.
    dense_layer_spec = get_gpt_layer_local_spec(
        num_experts=None,
        moe_grouped_gemm=False,
        qk_layernorm=config.qk_layernorm,
        gated_linear_unit=config.gated_linear_unit,
        multi_latent_attention=config.multi_latent_attention,
        normalization=normalization,
        use_flash_attention=config.use_flash_attention,
        qk_l2_norm=qk_l2_norm,
        use_alltoall=config.use_alltoall,
        use_fused_mla=config.use_fused_mla,
    )

    moe_layer_spec = get_gpt_layer_local_spec(
        num_experts=config.num_moe_experts,
        moe_grouped_gemm=True,
        qk_layernorm=config.qk_layernorm,
        gated_linear_unit=config.gated_linear_unit,
        multi_latent_attention=config.multi_latent_attention,
        normalization=normalization,
        use_flash_attention=config.use_flash_attention,
        qk_l2_norm=qk_l2_norm,
        use_alltoall=config.use_alltoall,
        use_fused_mla=config.use_fused_mla,
    )
```

MindSpore Transformers Hugging Face模型迁移——模型适配

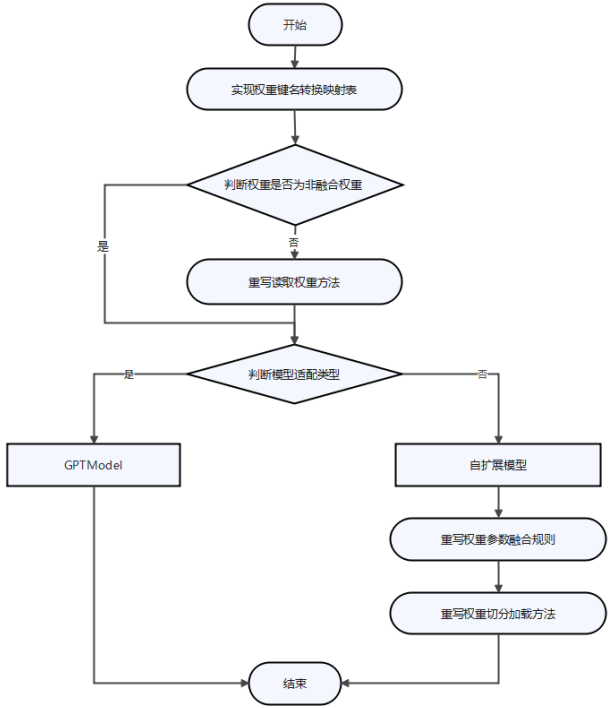
GPTModel作为MCore大语言模型统一入口，支持Transformer结构的模型构建能力，其init入参和调用示例如下

参数名	类型	描述	默认值
config	TransformerConfig	模型全局配置对象，包含隐藏层大小、注意力头数等核心参数	-
transformer_layer_spec	ModuleSpec	定义Transformer层结构的模块规范，指定SelfAttention/MLP等组件	-
vocab_size	int	词汇表大小，决定嵌入层和输出层的维度	-
max_sequence_length	int	序列最大长度，用于位置编码的初始化	-
pre_process	bool	是否执行嵌入层计算（当前仅支持True）	True
post_process	bool	是否计算输出层logits（默认启用）	True
parallel_output	bool	是否分散输出（保持张量并行分片）	True
share_embeddings_and_output_weights	bool	是否共享嵌入层与输出层权重（当前不支持）	False
position_embedding_type	str	位置编码类型	'learned_absolute'
rotary_percent	float	应用旋转编码的维度比例	1.0
rotary_base	int	RoPE基础值	10000
rope_scaling	bool	是否启用RoPE外推算法（通过rposition_embedding_type='llama3'控制）	False
seq_len_interpolation_factor	float	序列长度插值因子	None

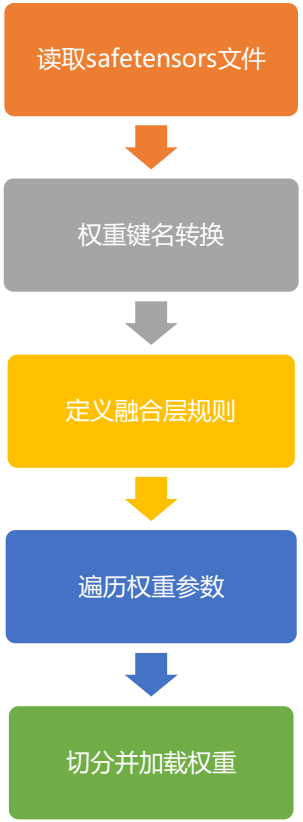
3. MindSpore Transformers Hugging Face权重加载

MindSpore Transformers 模型权重加载全流程

- 支持直接加载huggingface权重
- 可以处理多种泛化场景权重



MindSpore Transformers 模型权重加载全流程




```python
with safe_open(st_file, framework="np") as f:
# step1: 读取safetensors文件
    for name in f.keys():
        param = f.get_slice(name)
        # step2: 权重键名转换
        name = self.convert_name(name)
        yield name, param

# step3: 定义融合层规则
# 非MoE路由专家但需融合的权重键名部分映射规则:
[
    (".linear_qkv", ".linear_q", "q"),
    (".linear_qkv", ".linear_k", "k"),
    (".linear_qkv", ".linear_v", "v"),
    (".mlp.linear_fc1", ".mlp.gating", "gating"),
    (".mlp.linear_fc1", ".mlp.linear_fc1", "hidden"),
]

# MoE中与路由专家相关的权重键名部分映射规则:
[
    ("experts.weight1", "experts.{expert_id}.gating.", "{expert_id}","w1"),
    ("experts.weight1", "experts.{expert_id}.linear_fc1.", "{expert_id}","w3"),
    ("experts.weight2", "experts.{expert_id}.linear_fc2.", "{expert_id}","w2"),
]

# step4: 遍历权重参数
for name, loaded_weight in weights:
    if name in params_dict:
        param = params_dict[name]
        weight_loader = param.weight_loader
        # step5: 切分并加载权重
        weight_loader(param, loaded_weight, shard_id)
```

MindSpore Transformers Hugging Face模型权重加载

1. 在utils.py文件中，实现权重键名映射表。

元组左边为Hugging Face权重键名，右边为网络中的键名

```python
class Qwen3PreTrainedModel(PreTrainedModel, ModelMixin):
weight_mapping = [
        ('model.embed_tokens.', 'embedding.word_embeddings.'),
        ('.self_attn.q_proj.', '.self_attention.linear_q.'),
        ('.self_attn.k_proj.', '.self_attention.linear_k.'),
        ('.self_attn.v_proj.', '.self_attention.linear_v.'),
        ('.mlp.gate_proj.', '.mlp.gating.'),
        ('.mlp.down_proj.', '.mlp.linear_fc2.'),
        ('.mlp.up_proj.', '.mlp.hidden.'),
]
```

```python
# 获取网络中所有模块的键名
def get_params_dict(self):
    params_dict = {}
    for _, module in self.modules_dict.items():
        module_params = module.parameters_dict()
        for param_name, param in module_params.items():
            params_dict[param_name] = param

    return params_dict
```

MindSpore Transformers Hugging Face模型权重加载

- 提供统一接口_safetensors_weights_iterator 实现权重前处理
- 复用同一套权重加载流程，减少适配工作

2. 在modeling_xxx.py.py文件中，重写读取权重方法

将Hugging Face telechat2中已融合的权重进行拆分

```python
class InferenceTelechat2ForCausalLM(Telechat2PreTrainedModel, InferModelMixin):
    def _safetensors_weights_iterator(self, weights_files: List[str]) -> Generator[Tuple[str, Any], None, None]:
        """Iterate over the weights in the model safetensor files."""
        rank_id = get_tensor_model_parallel_rank()
        is_main_rank = rank_id == 0
        for st_file in tqdm(
                weights_files,
                desc=f"[Rank {rank_id}] Loading safetensors checkpoint shards",
                disable=not is_main_rank
        ):
            with safe_open(st_file, framework="np") as f:
                for name in f.keys():  # noqa: SIM118
                    # Return a lightweight PySafeSlice object
                    # that uses file pointer offset internally to read Safetensor
                    # on demand, avoiding memory explosion. Actual data can be obtained through slicing operation
                    # like param[start:end]
                    param = f.get_slice(name)
                    name = self.convert_name(name)
                    if ".key_value." in name and self.quant_config is None and self.config.quantization is None:
                        num_heads = self.config.n_head
                        n_kv_heads = self.config.num_key_value_heads
                        hidden_size = self.config.hidden_size
                        head_dim = hidden_size // num_heads
                        n_rep = num_heads // n_kv_heads
                        kv_channel = hidden_size // n_rep
                        param = np.array(param[:])
                        param = param.reshape((n_kv_heads, 2*head_dim, -1))
                        k_key = name.replace("key_value", "linear_k")
                        k_weight = param[:, :head_dim, :].reshape((kv_channel, -1))
                        v_key = name.replace("key_value", "linear_v")
                        v_weight = param[:, head_dim:2*head_dim, :].reshape((kv_channel, -1))
                        yield k_key, k_weight
                        yield v_key, v_weight
                    else:
                        yield name, param
```

MindSpore Transformers Hugging Face模型权重加载 --扩展

3. 重写参数融合规则

```
# 非MoE路由专家但需融合的权重键名部分映射规则:
[
    (".linear_qkv", ".linear_q", "q"),
    (".linear_qkv", ".linear_k", "k"),
    (".linear_qkv", ".linear_v", "v"),
    (".mlp.linear_fc1", ".mlp.gating", "gating"),
    (".mlp.linear_fc1", ".mlp.linear_fc1", "hidden"),
]
```

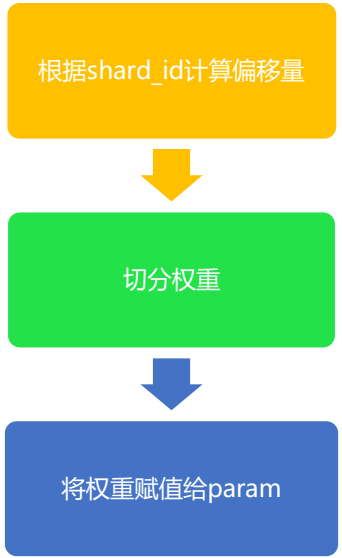
每一个元组中：
第一个元素是网络中融合层的权重名
第二个元素是Hugging Face权重转换后的权重键名
第三个元素是标志位，表示当前加载的权重是对应qkv中的哪一个，根据这个标志位进行后续权重切分与加载

```
# MoE中与路由专家相关的权重键名部分映射规则:
[
    ("experts.weight1", "experts.{expert_id}.gating.", "{expert_id}","w1"),
    ("experts.weight1", "experts.{expert_id}.linear_fc1.", "{expert_id}","w3"),
    ("experts.weight2", "experts.{expert_id}.linear_fc2.", "{expert_id}","w2"),
]
```

每一个元组中：
第一个元素是网络中融合层的权重名
第二个元素是Hugging Face权重转换后的权重键名
第三个元素是专家id，标识第几个专家
第四个元素是标志位，根据这个标志位进行后续权重切分与加载

MindSpore Transformers Hugging Face模型权重加载 --扩展

4. 重写模块的权重切分加载方法




```python
def weight_loader(self, param, loaded_weight, loaded_shard_id: Optional[str] = None):
        output_dim = getattr(param, "output_dim", None)
        tp_rank = self.tp_group.rank
        if loaded_shard_id is not None:
            if loaded_shard_id == "q":
                shard_offset = 0
                shard_size = self.num_heads * self.head_size
            elif loaded_shard_id == "k":
                shard_offset = self.num_heads * self.head_size
                shard_size = self.num_kv_heads * self.head_size
            elif loaded_shard_id == "v":
                shard_offset = (self.num_heads +self.num_kv_heads) * self.head_size
                shard_size = self.num_kv_heads * self.head_size
            shard_id = tp_rank
            start_idx = shard_id * shard_size
            # 权重在并行tp域切分
            loaded_weight = split_loaded_weight(loaded_weight, output_dim, start_idx, shard_size)
            param.asnumpy()[shard_offset:shard_offset + shard_size] = loaded_weight
```

4. MindSpore Transformers 模型迁移案例

Qwen系列模型迁移案例

Qwen2、Qwen3、Qwen3Moe目前已上线MindSpore Transformer，可参考对应modeling_infer.py代码实现

https://gitee.com/mindspore/mindformers/tree/master/mindformers/models/qwen2

https://gitee.com/mindspore/mindformers/tree/master/mindformers/models/qwen3

https://gitee.com/mindspore/mindformers/tree/master/mindformers/models/qwen3_moe

Qwen2

- 自注意力模块（multi_latent_attention=False）
- 前馈网络MLP模块（num_experts=None）
- RMSNorm模块（normalization="RMSNorm"）

Qwen3

- 自注意力模块（multi_latent_attention=False）
- 前馈网络MLP模块（num_experts=None）
- RMSNorm模块（normalization="RMSNorm"）
- qk norm结构（qk_layernorm=True）

Qwen3 Moe

- 自注意力模块（multi_latent_attention=False）
- MoE模块（num_experts=128）
- RMSNorm模块（normalization="RMSNorm"）
- qk norm结构（qk_layernorm=True）

```python
class InferenceQwen2ForCausalLM(Qwen2PreTrainedModel, InferModelMixin):
    r"""
    Provide qwen2 model infer through network.

    Args:
        config (Qwen2Config): The config of qwen2 model.

    Returns:
        output: Tensor, the output of qwen2 decoder layer

    """

    def __init__(self, config: Qwen2Config):
        super().__init__(config, auto_prefix=False)
        self.config = config
        config: TransformerConfig = self.convert_to_transformer_config(self.config)

        self.pad_token_id = self.config.pad_token_id
        self.vocab_size = config.vocab_size
        self.max_position_embeddings = config.max_position_embeddings
        self.compute_dtype = config.compute_dtype

        self.is_prefill = True
        self.model = GPTModel(config=config,
                              transformer_layer_spec=get_gpt_layer_local_spec(
                                  normalization=config.normalization,
                                  use_flash_attention=self.config.use_flash_attention,
                                  qk_layernorm=False,
                              ),
                              vocab_size=self.vocab_size,
                              max_sequence_length=self.max_position_embeddings,
                              position_embedding_type=config.position_embedding_type,
                              rotary_base=self.config.rope_theta,
                              share_embeddings_and_output_weights=self.config.tie_word_embeddings,
                              pre_process=self.config.pre_process,
                              post_process=self.config.post_process,)
```

MindSpore Transformers大模型使能教程：大模型Mcore内测教程

Qwen系列模型迁移按例

Qwen2、Qwen3、Qwen3Moe目前已上线MindSpore Transformer，可参考对应modeling_infer.py代码实现

https://gitee.com/mindspore/mindformers/tree/master/mindformers/models/qwen2

https://gitee.com/mindspore/mindformers/tree/master/mindformers/models/qwen3

https://gitee.com/mindspore/mindformers/tree/master/mindformers/models/qwen3_moe

Qwen2

- 自注意力模块（multi_latent_attention=False）
- 前馈网络MLP模块（num_experts=None）
- RMSNorm模块（normalization="RMSNorm"）

Qwen3

- 自注意力模块（multi_latent_attention=False）
- 前馈网络MLP模块（num_experts=None）
- RMSNorm模块（normalization="RMSNorm"）
- qk norm结构（qk_layernorm=True）

Qwen3 Moe

- 自注意力模块（multi_latent_attention=False）
- MoE模块（num_experts=128）
- RMSNorm模块（normalization="RMSNorm"）
- qk norm结构（qk_layernorm=True）

```python
class InferenceQwen3ForCausalLM(Qwen3PreTrainedModel, InferModelMixin):
    r"""
    Provide qwen3 model infer through network.

    Args:
        config (Qwen3Config): The config of qwen3 model.

    Returns:
        output: Tensor, the output of qwen3 decoder layer

    """

    def __init__(self, config: Qwen3Config):
        super().__init__(config, auto_prefix=False)
        self.config = config
        config: TransformerConfig = self.convert_to_transformer_config(self.config)

        self.quant_config = get_quant_config(self.config, self.weight_mapping)
        self.pad_token_id = self.config.pad_token_id
        self.vocab_size = config.vocab_size
        self.max_position_embeddings = config.max_position_embeddings
        self.compute_dtype = config.compute_dtype

        self.is_prefill = True
        self.model = GPTModel(config=config,
                              transformer_layer_spec=get_gpt_layer_local_spec(
                                  normalization=config.normalization,
                                  use_flash_attention=self.config.use_flash_attention,
                                  qk_layernorm=True,
                              ),
                              vocab_size=self.vocab_size,
                              max_sequence_length=self.max_position_embeddings,
                              position_embedding_type=config.position_embedding_type,
                              rotary_base=self.config.rope_theta,
                              share_embeddings_and_output_weights=self.config.tie_word_embeddings,
                              pre_process=config.pre_process,
                              post_process=config.post_process,
                              quant_config=self.quant_config)
```

MindSpore Transformers大模型使能教程：大模型Mcore内测教程

Qwen系列模型迁移实践

Qwen2、Qwen3、Qwen3Moe目前已上线MindSpore Transformer，可参考对应modeling_infer.py代码实现

https://gitee.com/mindspore/mindformers/tree/master/mindformers/models/qwen2

https://gitee.com/mindspore/mindformers/tree/master/mindformers/models/qwen3

https://gitee.com/mindspore/mindformers/tree/master/mindformers/models/qwen3_moe

Qwen2

- 自注意力模块（multi_latent_attention=False）
- 前馈网络MLP模块（num_experts=None）
- RMSNorm模块（normalization="RMSNorm"）

Qwen3

- 自注意力模块（multi_latent_attention=False）
- 前馈网络MLP模块（num_experts=None）
- RMSNorm模块（normalization="RMSNorm"）
- qk norm结构（qk_layernorm=True）

Qwen3 Moe

- 自注意力模块（multi_latent_attention=False）
- MoE模块（num_experts=128）
- RMSNorm模块（normalization="RMSNorm"）
- qk norm结构（qk_layernorm=True）

```python
class InferenceQwen3MoeForCausalLM(Qwen3MoePreTrainedModel, InferModelMixin):
    r"""
    Provide qwen3_moe model infer through network.

    Args:
        config (Qwen3MoeConfig): The config of qwen3_moe model.

    Returns:
        output: Tensor, the output of qwen3_moe decoder layer

    """

    def __init__(self, config: Qwen3MoeConfig):
        super().__init__(config, auto_prefix=False)
        self.config = config
        config: TransformerConfig = self.convert_to_transformer_config(self.config)

        # update communication-related configuration in TransformerConfig
        config = update_comm_config(config)
        self.pad_token_id = self.config.pad_token_id
        self.vocab_size = config.vocab_size
        self.max_position_embeddings = config.max_position_embeddings
        self.compute_dtype = config.compute_dtype
        self.is_prefill = True
        self.model = GPTModel(config=config,
                              transformer_layer_spec=get_gpt_layer_local_spec(
                                  num_experts=config.num_moe_experts,
                                  normalization=config.normalization,
                                  use_flash_attention=self.config.use_flash_attention,
                                  qk_layernorm=True,
                                  use_alltoall=config.use_alltoall,
                              ),
                              vocab_size=self.vocab_size,
                              max_sequence_length=self.max_position_embeddings,
                              position_embedding_type=config.position_embedding_type,
                              rotary_base=self.config.rope_theta,
                              share_embeddings_and_output_weights=self.config.tie_word_embeddings,
                              pre_process=config.pre_process,
                              post_process=config.post_process,)
```

MindSpore Transformers大模型使能教程：大模型Mcore内测教程

谢谢观看！

MindSpore Transformers LLM推理迁移与实践

MindSpore Transformers 大模型系列课程